M1- Mathématiques Appliquées Paris-Saclay - ENSTA
Apprentissage statistique (APM_4STA3), 2025-2026
Christine.Keribin@universite-paris-saclay.fr, Justine Lebrun (SNCF)

Mini projet
à envoyer pour le 14 mai 2026 minuit au plus tard

Consignes

Le projet est à effectuer avec le logiciel R *exclusivement* et donne lieu à un compte-rendu *rédigé* à effectuer en *binôme*, possiblement appartenant à deux groupes de TDs différents.

- Le compte-rendu doit présenter un titre informatif, une introduction pour préciser la problématique étudiée et le plan rédigé du document, une conclusion.
- Le document rédigé doit inclure dans le corps du texte les figures qui vous semblent importantes et les résultats nécessaires.
- Tout résultat doit être *justifié* et toute figure *commentée*. La notation prendra en compte la clarté et le soin de la rédaction.
- Trois fichiers seront à téléverser sur l'activité devoir du moodle: le `pdf` du compte-rendu (qui peut être manuscrit puis photographié ou scanné), un fichier texte `.R` (pas `Rmd`) contenant les commandes, et un autre fichier texte contenant vos prédictions pour le type de musique de Partie II. Les fichiers seront nommés avec les noms du binôme: `NOM1-NOM2.pdf`, `NOM1-NOM2.R`, `NOM1-NOM2_test.txt`.

  Le pdf ne doit pas comporter de commandes R.
- Aucun retard ne sera admis.

Outils génératifs Les connaissances apportées par le cours et les TPs sont suffisantes pour faire le projet sans utiliser d'outils génératifs. Cependant, si vous en avez trouvé la nécessité pour vous aider dans votre travail, dédiez une section particulière en annexe de votre rapport précisant le ou les outils utilisés, expliquant comment vous les avez utilisés, pour quelles tâches. Vous insérerez certains prompts significatifs. Vous indiquerez comment vous avez vérifié la validité des informations générées; si vous avez détecté des erreurs dans les réponses de ces outils automatiques, préciser lesquelles. Si vous n'avez utilisé aucun outil pour aucune tâche dans la réalisation de votre travail, vous le préciserez en introduction.

Introduction

Le jeu de données `Music_2026.txt` est extrait d'un challenge de reconnaissance de genre de musique[1], voir sa description en annexe.

Partie I: Analyse non supervisée

On commence par étudier le jeu de données dans son ensemble.

1. Effectuer quelques analyses descriptives (univariée et bivariée) pour le caractériser. Quelle est la proportion de chacun des genres de musique ?

   Expliquer pourquoi il peut être judicieux d'appliquer une transformation `log` aux variables `PAR_SC_V` et `PAR_ASC_V`, et de supprimer les variables numérotées de 148 à 167.

[1] qui avait été organisé par http://tunedit.org; les données ne sont maintenant plus disponibles

Discuter le cas des variables très corrélées (par exemple avec une corrélation supérieure à .99, couples que vous listerez), et celui des variables `PAR_ASE_M`, `PAR_ASE_MV`, `PAR_SFM_M` et `PAR_SFM_MV`.

2. Représenter le jeu de données sur les deux premiers plans principaux.

   Ces plans permettent-ils de bien discriminer les observations?

3. Effectuer une classification hiérarchique avec la méthode de Ward, puis tracer la silhouette.

   Comparer avec la classification définie par la variable `GENRE` dont vous tracerez également la silhouette.

   La normalisation a-t-elle un impact sur les résultats?

4. Définir l'échantillon d'apprentissage de la façon suivante:

```
set.seed(103)
train = sample(c(TRUE,FALSE),n,rep=TRUE,prob=c(2/3,1/3))
```

Partie II: Classification binaire

Dans cette partie, on se restreint aux deux genres `Classical` et `Jazz`. Filtrer ces deux genres dans l'échantillon d'apprentissage et l'échantillon de test de la question 4 Partie I pour définir les dataframes d'apprentissage et de test de la classification binaire. Vérifier qu'ils contiennent respectivement 2851 et 1503 observations.

1. Définir le modèle de régression logistique, puis l'estimer dans les cas suivants:
   - ModT formé par toutes les variables que vous aurez retenues à la question 1.
   - Mod1 formé par toutes les variables significatives au niveau 5% dans ModT.
   - Mod2 formé par toutes les variables significatives au niveau 20% dans ModT.
   - ModAIC obtenu par sélection de variables *stepwise* (`stepAIC`) sur critère AIC à partir d'un modèle initial que vous définirez (il est possible que vous ayez le temps d'une petite sieste en attendant les résultats). Indiquer dans le fichier R la définition précise du modèle `Y~?+?+...` retenu.

2. Tracer les courbes ROC (fonctions `prediction` et `performance` du package `ROCR`) calculées sur l'échantillon d'apprentissage et sur l'échantillon de test pour le modèle ModT, la courbe de la règle parfaite et la courbe de la règle aléatoire.

   Superposer les courbes ROC de tous les autres modèles calculées sur l'échantillon de test.

   Calculer l'aire sous la courbe ROC pour chacun des modèles et l'afficher dans la légende.

   Quel modèle suggérez-vous de retenir? Vérifier son adéquation.

3. Quel peut être l'intérêt de la régression ridge dans cette étude?

   Utiliser la fonction `glmnet` du package `glmnet`, pour un paramètre de régularisation λ variant de 10^{10} à 10^{-2}. A quoi s'apparentent ces deux cas extrêmes?

   Interpréter le graphique tracé par la fonction `plot` appliquée à l'objet sorti par `glmnet`.

4. Définir le germe du générateur (`set.seed`), puis estimer le paramètre de régularisation par une validation croisée en 10 segments sur l'échantillon d'apprentissage en utilisant la fonction `cv.glmnet`. Expliquer l'algorithme, puis commenter son résultat.

   Calculer la performance de cette méthode.

5. Compléter la figure de la question 2 avec les résultats de la régression ridge.

6. Récapituler, pour chaque modèle défini, l'ensemble des indicateurs construits.

   Quelle méthode préconiser dans cette étude? Pouvez-vous estimer sa performance de généralisation?

   Estimer les genres des extraits du fichier `Music_2026_test.txt` et les enregistrer dans le fichier `NOM1-NOM2_test.txt`

   [*Bonus*] Vous pouvez aussi faire marcher votre imagination et proposer d'autres méthodes que celles de l'énoncé.

Partie III: Classification multinomiale nominale

On souhaite maintenant définir un classifieur à plus de deux niveaux. On considère donc qu'une observation de condition d'expérience x (vecteur ligne de dimension p incluant l'intercept) est générée suivant une loi multinomiale $Y_k \sim \mathcal{M}(n_k, \pi(\mathrm{x_k}))$ à $C = 5$ niveaux, soit $\sum_{c=1}^{C} \pi_C(\mathrm{x}) = 1$.

1. Montrer que la loi multinomiale est une famille exponentielle de loi de paramètre naturel $\eta = (\log(\pi_1/\pi_C), \ldots, \log(\pi_{C-1}/\pi_C))$.

   Il est donc possible de rentrer dans le cadre d'un modèle linéaire généralisé avec une fonction de lien vectorielle. Pour chaque composante $\eta^{(c)}$ de η, $c = 1, \ldots, C-1$ on définit le lien

   $$\eta^{(c)}(\mathrm{x}) = \log \frac{\pi_c(\mathrm{x})}{\pi_C(\mathrm{x})} = \mathrm{x}\theta^{(c)}.$$

   La fonction de lien vectorielle η ainsi définie est appelée `softmax` et le paramètre à estimer est $\theta = (\theta^{(1)}, \ldots, \theta^{(C-1)})$. Ce modèle prend une modalité de référence (ici la modalité C), qui est à la discrétion de l'utilisateur.

   Calculer $\pi_c(\mathrm{x})$ en fonction des régresseurs linéaires $\mathrm{x}\theta_1, \ldots, \mathrm{x}\theta_{C-1}$.

2. Ecrire la log-vraisemblance $\mathcal{L}_K(\theta)$ d'un échantillon de K observations indépendantes.

   Montrer que les composantes du gradient s'écrivent pour $j = 1, \ldots, p$ et $c = 1, \ldots, C-1$:

   $$\frac{\partial}{\partial \theta_j^{(c)}} \mathcal{L}_n(\theta) = \sum_{k=1}^{K} x_{kj}(y_{kc} - n_k \pi_{kc}(\mathrm{x_k}\theta^{(c)}),$$

   et celles du Hessien, pour $j, \ell = 1, \ldots, p$ et $c, d = 1, \ldots, C-1$:

   $$\frac{\partial^2}{\partial \theta_j^{(c)} \partial \theta_\ell^{(d)}} \mathcal{L}_n(\theta) = \sum_{k=1}^{K} x_{kj} x_{k\ell} \pi_{kc}(\mathbb{1}_{c=d} - \pi_{kd})$$

3. A la question *Coder en R l'algorithme d'estimation d'une régression multinomiale nominale en utilisant une méthode de Newton*, ChatGPT produit le code en annexe, avec une remarque suggérant de considérer une version IRLS beaucoup plus stable:

   🔁 **1) Version IRLS (Fisher Scoring)**

   L'IRLS remplace le Hessien observé par son espérance :

   $$\mathcal{I}(\beta) = \sum_i x_i x_i^\top \otimes W_i$$

   👉 Concrètement : même structure que Newton, mais **plus stable numériquement.**

   Que pensez-vous de cette réponse ici?

   Etes-vous d'accord avec toutes les lignes du code généré? Commenter le calcul du Hessien.

4. Estimer le modèle avec la fonction `nnet::multinom`, puis calculer les erreurs d'apprentissage et de test.
5. La fonction `multinom` fait appel à la fonction `nnet` (du même package), qui estime les paramètres d'un réseau de neurones. Représenter le réseau correspondant, puis faire directement appel à `nnet` et retrouver les résultats de `multinom`.
6. Expliquer pourquoi il n'est plus possible de tracer une unique courbe ROC. Tracer les cinq courbes ROC de la prédiction d'un niveau contre les autres. Commenter.

Annexe: description du jeu de données

A database of 60 music performers has been prepared for the competition. The material is divided into six categories: classical music, jazz, blues, pop, rock and heavy metal. For each of the performers 15-20 music pieces have been collected. All music pieces are partitioned into 20 segments and parameterized. The descriptors used in parametrization also those formulated within the MPEG-7 standard, are only listed here since they have already been thoroughly reviewed and explained in many studies.

The feature vector consists of 191 parameters, the first 127 parameters are based on the MPEG-7 standard, the remaining ones are cepstral coefficients descriptors and time-related dedicated parameters:

```
a) parameter 1: Temporal Centroid,
b) parameter 2: Spectral Centroid average value,
c) parameter 3: Spectral Centroid variance,
d) parameters 4-37: Audio Spectrum Envelope (ASE) average values in 34 frequency bands
e) parameter 38: ASE average value (averaged for all frequency bands)
f) parameters 39-72: ASE variance values in 34 frequency bands
g) parameter 73: averaged ASE variance parameters
h) parameters 74,75: Audio Spectrum Centroid - average and variance values
i) parameters 76,77: Audio Spectrum Spread - average and variance values
j) parameters 78-101: Spectral Flatness Measure (SFM) average values for 24 frequency bands
k) parameter 102: SFM average value (averaged for all frequency bands)
l) parameters 103-126: Spectral Flatness Measure (SFM) variance values for 24 frequency bands
m) parameter 127: averaged SFM variance parameters
n) parameters 128-147: 20 first mel cepstral coefficients average values
o) parameters 148-167: the same as 128-147
p) parameters 168-191: dedicated parameters in time domain based of the analysis of the
distribution of the envelope in relation to the rms value.
q) GENRE: Classical, Jazz
```

Annexe: Algorithme codé par ChatGPT

```
multinom_newton <- function(X, y, max_iter = 100, tol = 1e-6) {
  # X : matrice n x p (avec intercept si souhaité)
  # y : vecteur de labels dans {1,...,K}

  n <- nrow(X)
  p <- ncol(X)
  K <- length(unique(y))
  Km1 <- K - 1
```

```
# Initialisation des coefficients
beta <- matrix(0, nrow = p, ncol = Km1)

# Fonction softmax (avec catégorie de référence)
compute_probs <- function(X, beta) {
  eta <- X %*% beta  # n x (K-1)
  exp_eta <- exp(eta)
  denom <- 1 + rowSums(exp_eta)

  P <- exp_eta / denom
  pK <- 1 / denom
  cbind(P, pK)  # n x K
}

loglik <- function(P, y) {
  sum(log(P[cbind(1:n, y)]))
}

for (iter in 1:max_iter) {

  P <- compute_probs(X, beta)  # n x K

  # Gradient (vectorisé)
  grad <- matrix(0, nrow = p, ncol = Km1)

  for (k in 1:Km1) {
    yk <- as.numeric(y == k)
    grad[, k] <- t(X) %*% (yk - P[, k])
  }

  grad_vec <- as.vector(grad)

  # Hessien
  H <- matrix(0, nrow = p * Km1, ncol = p * Km1)
  for (i in 1:n) {
    xi <- matrix(X[i, ], ncol = 1)  # p x 1
    pi <- P[i, 1:Km1]  # sans la référence
    Wi <- diag(pi) - tcrossprod(pi)  # (K-1)x(K-1)
    Hi <- kronecker(Wi, xi %*% t(xi))
    H <- H - Hi
  }

  # Mise à jour Newton
  step <- solve(H, grad_vec)

  beta_vec <- as.vector(beta)
  beta_new <- beta_vec - step
  beta <- matrix(beta_new, nrow = p, ncol = Km1)

  # Convergence
  if (max(abs(step)) < tol) {
    cat("Convergence atteinte en", iter, "itérations\n")
    break
```

```
    }

    if (iter == max_iter) {
      cat("Attention : convergence non atteinte\n")
    }
  }

  list(beta = beta, probs = compute_probs(X, beta))
}
```